

期末模拟试题 A

一、(8 分)

数据独立性是借助数据库管理数据的一个显著优点，请解释何谓物理独立性？

答：

物理独立性是指用户程序与数据存储相互独立。具体解释如下：

定义与内涵：物理独立性是指用户的应用程序与存储在磁盘上的数据库中数据是相互独立的。数据在磁盘上的存储方式，包括组织结构、存取方法等，由数据库管理系统（DBMS）负责管理。

实现机制：当数据库的存储结构发生改变时，数据库管理员会对模式/内模式映像做相应调整，以确保模式保持不变。这样，用户的应用程序就无需进行修改，从而保证了数据与程序的物理独立性。

物理独立性的存在，使得数据的物理存储方式可以灵活调整，而不会影响应用程序的正常运行，这是数据库系统相较于文件系统的一个重要优势。

二、(8 分)

请给出自主存取控制的定义。

答：

自主存取控制方法（Discretionary Access Control, DAC）是一种基于主题定制的存取控制方式。在这种方式下，主题的拥有者可以自主地选择特定的用户或用户组授予某些权限。特别是，该方式使主题拥有者透过特定标签或时间参数给予他人必须的权限。

三、(8 分)

何谓静态转储？

答：

静态转储是指在系统中无运行事务时进行的转储操作。在这种转储方式下，数据库在转储开始时处于一致状态，并且在转储期间不允许对数据库进行任何存取或修改活动。静态转储的目的是确保转储的数据在转储结束时是完整的，从而保证数据的有效性。然而，这种转储方式会降低数据库的可用性，因为转储必须在用户事务结束后进行，新的事务必须等待转储结束才能执行。

四、(10 分)

什么是封锁粒度？并分析不同的封锁粒度对于并发性与系统开销的影响。

答：

封锁的数据目标可以是这样一些逻辑单元：数据库、表、记录、字段等，封锁数据目标的大小叫封锁粒度。封锁的粒度小，并发度高，但封锁机构复杂，系统开销大。反之，封锁的粒度大，并发度小，但封锁机构简单，系统开销小。

五、(14 分)

设有关系模式 $R(U, F)$ ，其中 $U = \{A, B, C, D, E, G\}$ ，

函数依赖集 $F = \{B \rightarrow G, CE \rightarrow B, C \rightarrow A, CE \rightarrow G, B \rightarrow D, C \rightarrow D\}$

(1) 求 F 的最小函数依赖集 F_{min} ，请给出求解的主要过程。(3 分)

答：

现在，我们应用这些步骤到给定的函数依赖集 $F = \{B \rightarrow G, CE \rightarrow B, C \rightarrow A, CE \rightarrow G, B \rightarrow D, C \rightarrow D\}$ 上：

- $B \rightarrow G$ ：这是关于 G 的唯一信息，所以它是必要的。
- $CE \rightarrow B$ ：这是关于 B 的唯一信息（不考虑 B 自己推导自己的情况），所以它是必要的。
- $C \rightarrow A$ ：这是关于 A 的唯一信息，所以它是必要的。
- $CE \rightarrow G$ ：由于 $CE \rightarrow B$ 和 $B \rightarrow G$ ，我们可以推导出 $CE \rightarrow G$ 。因此，这个函数依赖是冗余的，可以删除。
- $B \rightarrow D$ ：这是关于 D 的一个信息，但我们还有 $C \rightarrow D$ 。由于 $CE \rightarrow B$ 和 $C \rightarrow D$ ，我们可以推导出 $CE \rightarrow D$ ，但这并不使 $B \rightarrow D$ 冗余，因为 B 可以单独推导 D 。所以 $B \rightarrow D$ 是必要的。
- $C \rightarrow D$ ：同上，由于 $B \rightarrow D$ 的存在，这个函数依赖看似冗余，但实际上它是关于 D 的另一个独立来源，且不能由其他函数依赖推导出来（不考虑 $B \rightarrow D$ 的话）。但考虑到我们已经保留了 $B \rightarrow D$ ，并且 $CE \rightarrow B$ 可以推导出 B ，进而推导出 D ，所以 $C \rightarrow D$ 在这个上下文中是冗余的。

因此，最小函数依赖集 $F_{\min}$ 为：

$$F_{\min} = \{B \rightarrow G, CE \rightarrow B, C \rightarrow A, B \rightarrow D\}.$$

(2) 求出 R 的候选码，请给出求解过程。(3 分)

答：

为了求出关系模式 $R(U, F)$ 的候选码，我们需要按照以下步骤进行：

1. 确定必须包含的属性：

- 首先找出只在函数依赖的右边出现、而不在左边出现的属性，这些属性必须包含在候选码中。我们称这些属性为主属性。
- 在本题中，函数依赖集 $F = \{B \rightarrow G, CE \rightarrow B, C \rightarrow A, CE \rightarrow G, B \rightarrow D, C \rightarrow D\}$ 。
- 检查每个属性：
 - A 只在 $C \rightarrow A$ 的右边出现。
 - B 在多个依赖的左边出现。
 - C 在多个依赖的左边出现。
 - D 只在 $B \rightarrow D$ 和 $C \rightarrow D$ 的右边出现。
 - E 只在 $CE \rightarrow B$ 和 $CE \rightarrow G$ 的左边出现。
 - G 只在 $B \rightarrow G$ 和 $CE \rightarrow G$ 的右边出现。
- 因此， A 、 D 和 G 必须包含在候选码中，但由于 D 和 G 可由其他属性推出，我们主要关注 A 必须由候选码包含，且由于 $C \rightarrow A$ ， C 也必须在候选码中考虑。

2. 找出能唯一标识所有属性的属性集合：

- 通过闭包算法找出能推导出所有属性的最小属性集合。
- 首先考虑 (C, E) ：
 - $CE \rightarrow B$, 得到 B 。
 - $B \rightarrow G$, 得到 G 。
 - $B \rightarrow D$, 得到 D 。
 - $C \rightarrow A$, 得到 A 。
- 到此为止, 我们已经通过 (C, E) 推导出所有属性 $\{A, B, C, D, E, G\}$ 。

3. 验证最小性：

- 尝试去掉 C 或 E 看是否还能推导出所有属性：
 - 如果只有 E , 则无法推导出任何属性。
 - 如果只有 C ：
 - $C \rightarrow A$, 得到 A 。
 - $C \rightarrow D$, 得到 D 。
 - 但无法推导出 B, E, G 。
- 因此, (C, E) 是最小的属性集合, 能够唯一标识所有属性。

综上所述, 关系模式 $R(U, F)$ 的候选码是 $\{C, E\}$ 。

(3) 判断 R 中哪些属性是主属性, 哪些是非主属性? (2 分)

答：

在关系模式 $R(U, F)$ 中, 主属性是指那些出现在函数依赖左边的属性, 或者能够通过其他函数依赖间接推导出来的属性。非主属性则是指那些只出现在函数依赖右边的属性, 且不能通过其他属性推导出来。

给定 $U = \{A, B, C, D, E, G\}$ 和函数依赖集 $F = \{B \rightarrow G, CE \rightarrow B, C \rightarrow A, CE \rightarrow G, B \rightarrow D, C \rightarrow D\}$, 我们可以按照以下步骤判断哪些属性是主属性, 哪些是非主属性：

1. 找出所有函数依赖的左部属性：

- $B \rightarrow G$: 左部是 B
- $CE \rightarrow B$: 左部是 CE (包含 C 和 E)
- $C \rightarrow A$: 左部是 C
- $CE \rightarrow G$: 左部是 CE (包含 C 和 E , 但已经考虑过)
- $B \rightarrow D$: 左部是 B (已经考虑过)
- $C \rightarrow D$: 左部是 C (已经考虑过)

从上面的分析中, 我们可以看出 B, C, E 出现在函数依赖的左部。

2. 确定主属性：

- 直接出现在函数依赖左部的属性 B, C, E 是主属性。
- 由于 $CE \rightarrow B$, C 和 E 一起也能推导 B , 但这不改变 B 已经是主属性的事实。

3. 确定非主属性：

- 检查剩下的属性 A, D, G 是否只出现在函数依赖的右边。
- A 只出现在 $C \rightarrow A$ 的右边, 所以 A 是非主属性。
- D 只出现在 $B \rightarrow D$ 和 $C \rightarrow D$ 的右边, 所以 D 是非主属性。
- G 只出现在 $B \rightarrow G$ 和 $CE \rightarrow G$ 的右边, 所以 G 是非主属性。

综上所述, 关系模式 $R(U, F)$ 中的主属性是 B, C, E , 非主属性是 A, D, G 。

(4) 对 R 属于哪一类范式? 为什么? (3 分)

答:

为了判断关系模式 $R(U, F)$ 属于哪一类范式, 我们需要根据范式的定义来进行分析。

首先, 我们回顾范式的定义:

1. 第一范式 (1NF): 关系模式中的每个属性都是原子的, 即不可再分的。
2. 第二范式 (2NF): 在1NF的基础上, 关系模式中的非主属性完全依赖于整个候选码。
3. 第三范式 (3NF): 在2NF的基础上, 关系模式中的非主属性不传递依赖于候选码。即, 如果 $X \rightarrow Y$ 且 $Y \subseteq X$, 且 $Y \rightarrow Z$, 则 $X \rightarrow Z$ 不应该成立 (除非 Z 是 X 的子集或 X 是候选码)。

现在, 我们来分析关系模式 $R(U, F)$:

- $U = \{A, B, C, D, E, G\}$
- $F = \{B \rightarrow G, CE \rightarrow B, C \rightarrow A, CE \rightarrow G, B \rightarrow D, C \rightarrow D\}$

首先, R 显然是1NF的, 因为题目没有提及任何属性是可分的。

接下来, 我们找出候选码。通过之前的分析或计算, 我们可以确定候选码是 $\{C, E\}$ (因为 CE 能够唯一标识关系中的每一个元组, 并且没有更小的属性集能够做到这一点)。

然后, 我们检查是否满足2NF:

- 非主属性 A, D, G 是否完全依赖于候选码 $\{C, E\}$?
 - A 依赖于 C ($C \rightarrow A$), 而 C 是候选码的一部分, 所以满足2NF。
 - D 依赖于 B ($B \rightarrow D$) 和 C ($C \rightarrow D$), 但 B 不是候选码, 所以这里存在问题, 不过我们会在3NF的分析中进一步讨论。
 - G 依赖于 B ($B \rightarrow G$), 而 B 可以通过候选码 $\{C, E\}$ 推导出来 ($CE \rightarrow B$), 所以初步看似满足2NF, 但我们需要进一步考虑3NF。

接着, 我们检查是否满足3NF:

- 是否存在非主属性传递依赖于候选码?
 - $B \rightarrow G$ 和 $CE \rightarrow B$ 一起构成了传递依赖, 因为 $CE \rightarrow B$ 且 $B \rightarrow G$, 但 $CE \nrightarrow G$ 不是直接依赖 (尽管 $CE \rightarrow G$ 在函数依赖集中, 但这不影响传递依赖的判断)。这里的关键是 B 不是候选码, 而是可以通过候选码推导出来的属性。
 - 类似地, $B \rightarrow D$ 也构成了传递依赖, 因为 $CE \rightarrow B$ 且 $B \rightarrow D$ 。

由于存在传递依赖, 所以 R 不满足3NF。

综上所述, 关系模式 $R(U, F)$ 属于第二范式 (2NF), 但不满足第三范式 (3NF)。这是因为存在非主属性 G 和 D 传递依赖于候选码 $\{C, E\}$ 通过属性 B 。为了将 R 分解到3NF, 我们需要消除这些传递依赖, 通常通过创建新的关系模式来实现, 使得每个非主属性都直接依赖于其所在关系模式的候选码。

(5) 若 R 未达到 3NF, 请模式分解 R 以达到 3NF, 并保持函数依赖性。 (3 分)

答:

为了将关系模式 $R(U, F)$ 分解到第三范式 (3NF) 并保持函数依赖性, 我们需要首先识别出违反 3NF 的函数依赖, 并据此对关系模式进行分解。

在 $R(U, F)$ 中, $U = \{A, B, C, D, E, G\}$, 函数依赖集 $F = \{B \rightarrow G, CE \rightarrow B, C \rightarrow A, CE \rightarrow G, B \rightarrow D, C \rightarrow D\}$ 。

我们注意到, 存在以下函数依赖:

- $B \rightarrow G$
- $B \rightarrow D$

这两个依赖表明, 非主属性 G 和 D 都依赖于非候选码属性 B 。由于 B 不是候选码 (候选码是 $\{C, E\}$), 这违反了 3NF 的要求, 即非主属性不应传递依赖于候选码, 也不应直接依赖于非候选码属性 (除非该非候选码属性本身是某个候选码的一部分, 但在这里 B 不是)。

为了分解 R 以达到 3NF, 我们可以考虑将依赖于 B 的属性分离出来, 形成一个新的关系模式。同时, 我们需要确保新的关系模式之间保持函数依赖性。

一种可能的分解方法是:

1. 创建两个新的关系模式 $R_1(U_1, F_1)$ 和 $R_2(U_2, F_2)$ 。

- $R_1(U_1, F_1)$: 包含候选码 $\{C, E\}$ 和所有直接依赖于它们的属性。在这里, $U_1 = \{C, E, A\}$, $F_1 = \{C \rightarrow A, CE \rightarrow B\}$ (注意, 我们保留了 $CE \rightarrow B$ 以便在需要时能够恢复 B 的值)。
- $R_2(U_2, F_2)$: 包含属性 B 和所有直接依赖于它的属性。在这里, $U_2 = \{B, G, D\}$, $F_2 = \{B \rightarrow G, B \rightarrow D\}$ 。

2. 验证分解后的关系模式是否满足 3NF:

- R_1 : 在 R_1 中, 所有非主属性 (即 A) 都直接依赖于候选码 $\{C, E\}$ 的一部分 (即 C), 因此满足 3NF。
- R_2 : 在 R_2 中, 所有非主属性 (即 G 和 D) 都直接依赖于主属性 B , 因此也满足 3NF。

3. 验证分解是否保持函数依赖性:

- 原始函数依赖集 F 中的所有函数依赖都可以通过分解后的关系模式 R_1 和 R_2 中的函数依赖来推导出来。例如, 要推导 $CE \rightarrow G$, 我们可以先在 R_1 中使用 $CE \rightarrow B$, 然后在 R_2 中使用 $B \rightarrow G$ 。

因此, 通过上述分解, 我们得到了两个满足 3NF 的关系模式 $R_1(C, E, A)$ 和 $R_2(B, G, D)$, 并且这两个关系模式之间保持了原始的函数依赖性。

六、(16 分)

一、已知某部门的销售系统, 其部分关系模式如下:

- Orders(OrderID, CustomerID, OrderDate, RequiredDate, ShippedDate);

含义: 订单表(订单编号, 客户编号, 订购日期, 预计到达日期, 发货日期);

- OrderDetails(OrderID, ProductID, UnitPrice, Quantity)

含义: 订单明细表(订单编号, 产品编号, 单价, 订购数量);

- Products(ProductID, ProductName, SupplierID, CategoryID, UnitsInStock)

含义: 产品表(产品编号, 产品名称, 供应商编号, 类型编号, 库存数量)

- Customers(CustomerID, CompanyName, ContactName, ContactTitle, Address, Phone)

含义: 客户表(客户编号, 所在公司名称, 客户姓名, 客户头衔, 联系地址, 电话)

请完成下面的 SQL 查询:

(1) 查询订单编号为 “10248” 的订单的发货日期。(4 分)

SELECT ShippedDate

```
FROM Orders
WHERE OrderID = '10248';
```

(2) 查询各个订单的订单金额，列标题显示为:订单编号， 订单金额。(4 分)

答: 为了查询各个订单的订单金额，并显示订单编号和对应的订单金额，我们需要结合 **Orders** 表和 **OrderDetails** 表。**Orders** 表提供了订单的框架，而 **OrderDetails** 表则详细列出了每个订单中的产品、单价以及数量。订单金额是通过将 **OrderDetails** 表中的每个产品的 **UnitPrice** (单价) 乘以 **Quantity** (数量)，然后对同一个 **OrderID** (订单编号) 下的所有产品进行求和得到的。

```
SELECT
    Orders.OrderID AS 订单编号,
    SUM(OrderDetails.UnitPrice * OrderDetails.Quantity) AS 订单金额
FROM
    Orders JOIN OrderDetails ON Orders.OrderID = OrderDetails.OrderID
GROUP BY
    Orders.OrderID;
```

(3) 更新订单明细表，将库存量少于 500 的产品的单价上调 10%。(4 分)

答: 为了更新订单明细表 (**OrderDetails**) 中库存量少于 500 的产品的单价，我们首先需要确定哪些产品的库存量少于 500。这一信息存储在 **Products** 表中。接着，我们需要根据这些产品的 **ProductID** 来找到 **OrderDetails** 表中对应的记录，并将它们的 **UnitPrice** 上调 10%。

以下是一个 SQL 查询语句，用于实现这一更新操作：

```
UPDATE OrderDetails
SET UnitPrice = UnitPrice * 1.10
WHERE ProductID IN (
    SELECT ProductID
    FROM Products
    WHERE UnitsInStock < 500
);
```

(4) 查询这样订单的订单编号：同一个订单同时购买了所有产品类型编号为“DT” 的产品。(4 分)

答: 要查询同时购买了所有产品类型编号为“DT” 的产品的订单编号，我们需要确保订单中的产品明细包含了所有“DT” 类型的产品。这通常是一个比较复杂的查询，因为它涉及到对多个表的连接、分组、聚合以及子查询的使用。

以下是一个可能的 SQL 查询方案，用于找出同时购买了所有“DT” 类型产品的订单编号：
-- 首先，找出所有类型为“DT” 的产品编号

```
WITH DT_Products AS (
    SELECT ProductID
    FROM Products
```

```

        WHERE CategoryID = 'DT'
    ),
    -- 然后，找出每个订单中包含的“DT”类型产品数量
    Order_DT_Product_Counts AS (
        SELECT O.OrderID, COUNT(DISTINCT OD.ProductID) AS DT_Product_Count
        FROM Orders O
        JOIN OrderDetails OD ON O.OrderID = OD.OrderID
        JOIN DT_Products DTP ON OD.ProductID = DTP.ProductID
        GROUP BY O.OrderID
    ),
    -- 接着，计算总共有多少种“DT”类型的产品
    Total_DT_Products AS (
        SELECT COUNT(*) AS Total_Count
        FROM DT_Products
    )
    -- 最后，找出那些订单中包含的“DT”类型产品数量等于总“DT”类型产品数量的订单
    SELECT OPC.OrderID
    FROM Order_DT_Product_Counts OPC
    CROSS JOIN Total_DT_Products TDP
    WHERE OPC.DT_Product_Count = TDP.Total_Count;

```

解释：

DT_Products CTE（公用表表达式）：这个部分查询出所有类型为“DT”的产品编号。

Order_DT_Product_Counts CTE：这个部分计算出每个订单中包含的不同“DT”类型产品的数量。我们使用了 DISTINCT 来确保不会重复计算同一个产品。

Total_DT_Products CTE：这个部分计算出总共有多少种不同的“DT”类型产品。

最终查询：我们将 Order_DT_Product_Counts 和 Total_DT_Products 进行 CROSS JOIN（交叉连接），因为这两个结果集都是单行的（或者应该是），然后我们筛选出那些订单中包含的“DT”类型产品数量等于总“DT”类型产品数量的订单。

七、（12 分）

设有一个金融数据库，包括 client、bcard、insurance、property 4 个关系模式：

client（客户）表

字段名称	说明
cnum	客户编码PRIMARY KEY
cname	客户名称
cphone	客户电话
cpassword	客户登录密码
cage	客户年龄
cgender	客户性别

bcard（银行卡）表

字段名称	说明
bnum	银行卡号PRIMARY KEY
btype	银行卡类型
cnum	所属客户编号 注：FOREIGN KEY引用自client表的 cnum 字段。

insurance（保险）表

字段名称	说明
iname	保险名称
inum	保险编号PRIMARY KEY
iamount	保险金额
iperson	适用人群
iyear	保险年限
iproject	保障项目

property（资本）表

字段名称	说明
id	资产编号PRIMARY KEY
cnum	客户编号 说明：FOREIGN KEY引用自client表的cnum字段。
pifid	商品编号 说明：FOREIGN KEY引用自insurance表的inum字段。
quan	商品数量
ptime	购买时间

用关系代数完成如下查询：

（1）查询购买了 10 份保险（商品数量为 10）的客户编号；（4 分）

$\pi_{Client.cnum}(\sigma_{iamount = 10}(insurance))$

（2）查询没有购买适用于儿童的保险的客户编号；（4 分）

$\pi_{Client.cnum}(client) - \pi_{Client.cnum}(\sigma_{iperson = 'Children'}(insurance \bowtie property))$

(3) 查询至少拥有客户编号为 001 所拥有银行卡类型的客户姓名和电话。(4 分)

$\pi_{\text{name, cphone}}(\text{client} \bowtie (\pi_{\text{cnum, btype}}(\text{bcard}) \bowtie (\sigma_{\text{cnum}=001}(\text{bcard}) \times \text{client})))$

八、(12 分)

设有一个 SPJ 数据库，包括 S、P、J 和 SPJ 共 4 个关系模式：

1. 供应商表由供应商代码 SNO、供应商姓名 SNAME、供应商状态 STATUS、供应商所在城市 CITY 组成：S(SNO, SNAME, STATUS, CITY) 主码：SNO
2. 零件表由零件代码(PNO)、零件名(PNAME)、颜色(COLOR)、重量(WEIGHT)组成：P(PNO, PNAME, COLOR, WEIGHT) 主码：PNO
3. 工程项目表 J 由工程项目代码(JNO)、工程项目名(JNAME)、工程项目所在城市(CITY)组成：J(JNO, JNAME, CITY) 主码：JNO
4. 供应情况表 SPJ 由供应商代码 SNO、零件代码(PNO)、工程项目代码(JNO)、供应数量(QTY)组成：SPJ(SNO, PNO, JNO, QTY)
主码：(SNO, PNO, JNO)
外码：SNO，引用了供应商表的供应商代码 SNO
PNO，引用了零件表的零件代码 PNO
JNO，引用了工程项目表的工程项目代码 JNO

用关系代数完成如下查询：供应一汽工程（工程项目名为‘一汽’）红色零件（零件的颜色为‘红’）的供应商代码 SNO。

(1) 画出上述查询的关系代数语法树：(4 分)

(2) 对题 1 中的关系代数语法树进行优化，画出优化后的语法树。(8 分)

首先，我们来构建关系代数查询，以找出供应一汽工程（工程项目名为‘一汽’）红色零件（零件的颜色为‘红’）的供应商代码 SNO。

关系代数查询可以表示如下：

```
text Copy Code
1   $\pi_{\text{SNO}}(\sigma_{\text{JNAME} = \text{'一汽'} \wedge \text{COLOR} = \text{'红'}}(\text{SPJ} \bowtie (\text{J} \bowtie (\text{P} \bowtie \text{S}))))$ 
```

但是，这个查询可以进一步简化，因为我们不需要在所有关系之间都执行笛卡尔积。我们可以按照以下步骤来构建更优化的查询：

1. 首先，从 J 关系中选择工程项目名为‘一汽’的元组：

```
text Copy Code
1   $\text{J1} := \sigma_{\text{JNAME} = \text{'一汽'}}(\text{J})$ 
```

2. 然后，从 P 关系中选择颜色为‘红’的元组：

```
text Copy Code
1   $\text{P1} := \sigma_{\text{COLOR} = \text{'红'}}(\text{P})$ 
```

3.接下来, 我们需要将 SPJ 关系与 J1 和 P1 进行连接, 以找出供应一汽工程红色零件的供应商代码。这可以通过两次自然连接来实现:

```
text                                                                    Copy Code
1  SPJ1 := SPJ ⋈ J1
2  SPJ2 := SPJ1 ⋈ P1
```

4.最后, 从 SPJ2 中投影出供应商代码SNO:

```
text                                                                    Copy Code
1   $\pi$  SNO (SPJ2)
```

现在, 我们来画出关系代数语法树:

原始语法树 (未优化):

```
text                                                                    Copy Code
1       $\pi$  SNO
2      |
3       $\sigma$  JNAME = '一汽'  $\wedge$  COLOR = '红'
4      |
5      ⋈
6      /  \
7     SPJ  ⋈
8          /  \
9         J   ⋈
10         /  \
11        P   S
```

优化后的语法树:

```
text                                                                    Copy Code
1       $\pi$  SNO
2      |
3      ⋈ (P1)
4      /  \
5     ⋈ (J1)  P1:  $\sigma$  COLOR = '红' (P)
6      /  \
7     SPJ  J1:  $\sigma$  JNAME = '一汽' (J)
```

在优化后的语法树中, 我们首先分别选择了满足条件的 J 和 P 关系 (即 J1 和 P1), 然后将它们与 SPJ 关系进行连接, 并最终投影出供应商代码SNO。这样的查询更加高效, 因为它避免了不必要的笛卡尔积操作, 并且只处理了满足条件的数据。

九、（12 分）

为某公司建立一个管理数据库，要求如下：

- ①该公司有多个部门，各部门有许多员工，但每一个员工仅属于一个部门；
- ②每个部门可以管理多个工程，每个工程由唯一的部门负责管理；
- ③每个员工可在多项工程中工作，工种可以是普通工作或管理工作等，每项工程可有多个员工做工；

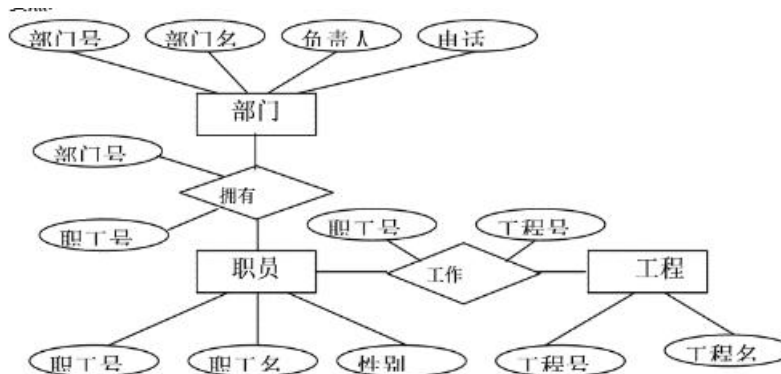
④“部门”有属性：部门号、部门名、部门负责人、电话；

“员工”有属性：员工号，员工名，性别；

“工程”有属性：工程号，工程名，预算

根据以上要求，完成：

- （1）画出 E-R 图，并注明属性和联系类型（6 分）。
- （2）将 E-R 图转换成关系模型，并注明主码和外码（6 分）。



下面给出已经转换好的等价关系模型结构。

- ①部门（部门号，部门名，部门负责人，电话）（主关键字为部门号）
- ②职员（职工号，职工名，性别，部门号）（主关键字为职工号）
- ③工程（工程号，工程名，项目负责人）（主关键字为工程号）
- ④工作（职工号，工程号，工种）[主关键字为（职工号，工程号）]